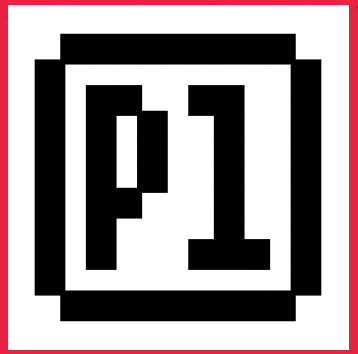
Compilación



Martín René Vilugrón
mrvilugron@unrn.edu.ar

UNRN
Universidad Nacional
de Río Negro

07 / 2026

r33

La traducción

UNRN Universidad Nacional de Río Negro

Aunque para compilar

```
$> gcc archivo.c
```

Y para ejecutar, termina siendo



./a.exe



./a.out

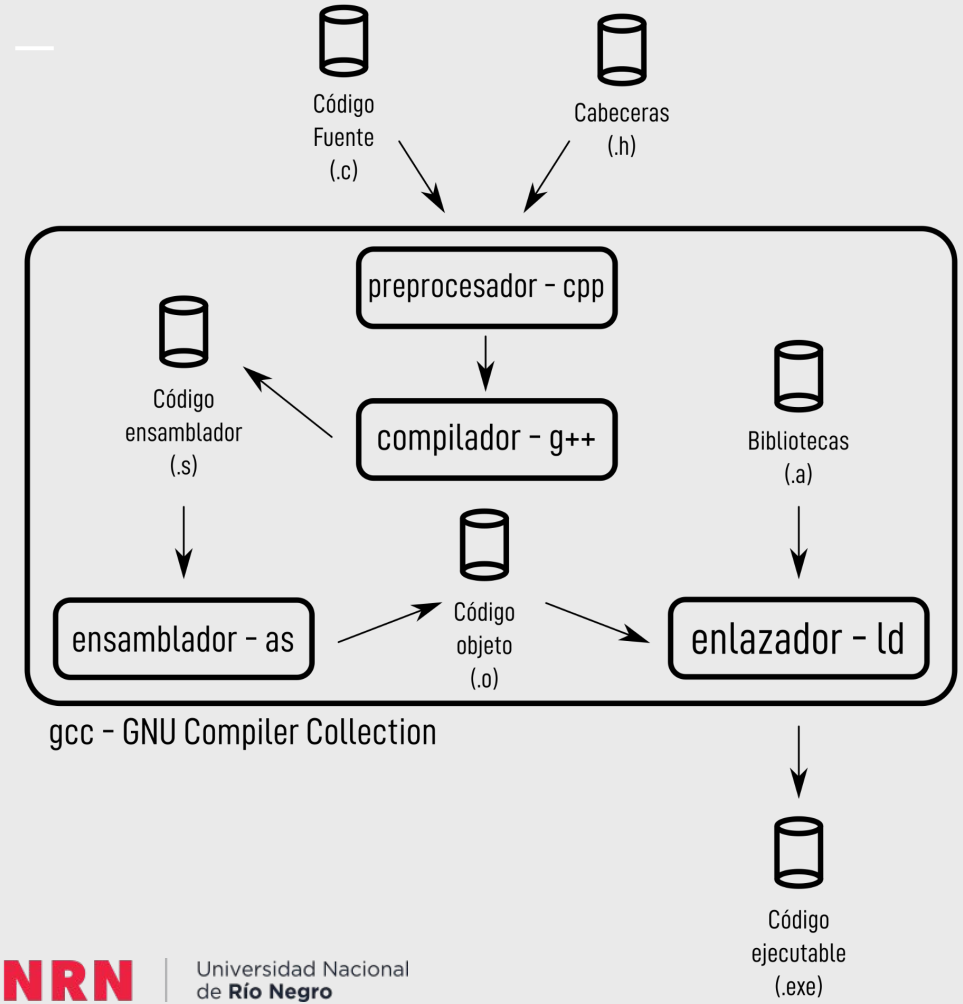
El paso de código a instrucciones no es directo

Pero...
¿qué pasa cuando compilamos?

¿¡expecto binarium?!



¡Solo un conjunto de programas!



Es posible llamar a cada programa individualmente

Pero si hacemos:

```
$> gcc --verbose programa.c
```

...vemos por qué no es viable

¡A la terminal!



Fase 1 - Preprocesador

- Elimina los comentarios
- ***Resuelve directivas***

`#define origen` ***destino***

Crea sustituciones textuales,
origen con destino
¿En lo que sea?

Ejemplos de #define

Los usos más habituales

```
#define PI 3.14159
#define ARREGLO_MAX 10
#define ERROR_DIVISION_POR_CERO -1

// Como constantes, pero más constantes
```

Ayuda a que el código sea más explícito

```
int resultado = division_positiva(dividendo, divisor);

if (resultado == ERROR_DIVISION_POR_CERO)
{
  printf("¯\_(ツ)_/¯");
}
else if (resultado == ERROR_DIVISION_NEGATIVA)
{
  printf("Error en los argumentos");
}
```

Lo podemos definir al compilar

```
$> gcc -DNOMBRE_MACRO
# o, si va con argumentos
$> gcc -DOTRO_MACRO=20
```

Pero si ya está definido en el archivo, este se ignora.

Ojo que le podemos dar ¡*muy mal uso*!

```
#define + *
#define si if
```

Esto es *antes* del compilador.

¡A la terminal!



{ Existe una substitución con parámetros, pero reduce *mucho* la legibilidad }

Compilación condicional
#ifdef / #ifndef / #else

Permite incluir o excluir código de la compilación en función de una condición

Esto es la base de la Fase 2 de las prácticas

Un uso interesante

```
#ifdef DEBUG
  printf("Debug: El valor de x es %d\n", x);
#endif
```

Se lee: *Si el símbolo **`DEBUG`** está definido, el `printf` vá.*

```
$> gcc -DDEBUG programa.c
```

Qué combinado con la llamada al compilador es súper útil.

Pero es la base de la compilación separada y las librerías

```
#ifndef MI_CABECERA_H
#define MI_CABECERA_H

// Contenido del header;
// - Constantes como MACROS
// - prototipos públicos más su documentación
// - variables globales compartidas (más globales aún)

#endif // MI_CABECERA_H
```

En un archivo `mi_cabecera.h`

Por ejemplo, `arreglos.h`

```
#ifndef ARREGLOS_H
#define ARREGLOS_H

/*
 * Suma los elementos del arreglo
 */
int suma(size_t largo, int arreglo[]);

#endif // ARREGLOS_H
```

Inclusión textual

```
#include "archivo"
#include <archivo>
```

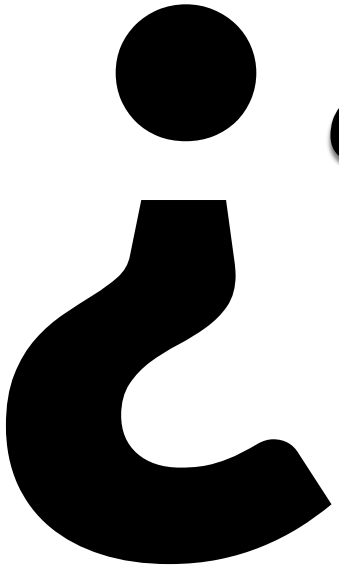
Trae el contenido de archivo al archivo actual

Le indicamos dónde buscar

```
// Para las librerías ‘de sistema’ la STDLIB y lo instalado
#include <stdio.h>

// para las librerías ‘propias’, al lado del .c
#include "arreglos.h"
```

Podemos agregar archivos a ambos lugares con opciones al compilador



Pero ¿para qué está la compilación condicional mezclada acá?



{ }

Para evitar problemas de

re-definición de tipos

(en cosas que vienen más adelante)

¡A la terminal!




¿Preguntas?

Fase 2 - Compilación

Aquí se transforma el código C a Assembler (.s)

(esto ya es 'dependiente' de la computadora objetivo)

https://godbolt.org/z/99r71E13q

Optimización

El compilador busca oportunidades para mejorar el rendimiento de nuestro código sin cambiar su significado

Con las opciones `-O1` `-O2` `-O3`

{ Esta es una mínima lista de lo que puede hacer el compilador. }

Plegado de constantes

1

En una instrucción como

```
int segundos_por_dia = 24 * 60 * 60;
```

el compilador calcula el resultado (`86400`) directamente y pone ese valor en el código ensamblador.

Plegado de constantes

```
int segundos_por_dia = 24 * 60 * 60;
```

Queda como

```
int segundos_por_dia = 86400;
```

Poda de código muerto

2

Si hay una instrucción que nunca se ejecuta, como lo que esta luego de un `return` o un `break`.

El compilador lo poda ya que no es necesario

Lo que esta despues del retorno

```
int funcion()
{
  return;
  for(int i = 0; i < 10;i++)
  {
    printf("%d, ", i);
  }
}
```

Prueben con `gcc -S optimiza.c` **y** `gcc -O3 -S optimiza.c`

Expansión de lazos

3

Si el código siempre va hasta el mismo valor, y el mismo es de un tamaño reducido, el compilador puede suprimir el lazo completamente

Uso de instrucciones especializadas

4

En algunos casos y dependiendo de la computadora, esta cuenta con instrucciones especializadas que hacen más cosas con menos (SIMD).
Las operaciones de arreglos son el principal

{ Igual tienen una materia dedicada a estudiar compiladores ;-) }

Fase 3 - ensamblado

Aquí se transforma el Assembler a código objeto (.o) en un formato binario.

Fase 4 - enlace

Es la construcción final del programa, tomando el código objeto (.o) y las librerías (.a) para ensamblar el ejecutable

¿Qué se une?

Aquí se toman nuestro código objeto y se lo unen con las librerías compiladas disponibles, como la STDLIB.

Que se busca conectar

Todos los “símbolos” deben de estar enlazados a algo.

Estos pueden ser funciones pero tambien variables globales.

Fase 5 - ejecución

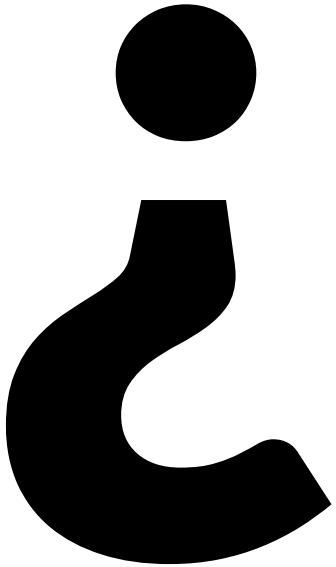
Finalmente, tenemos nuestro programa para utilizar, podemos indicarle al compilar que nombre tendrá con la opción

```
-o nombre
```

Lo que nos trae denuevo a...

UNRN Universidad Nacional de Río Negro

headers (.h)



Qué son



Headers

Se usan para agrupar lo que es ‘compartible’ con otras partes del programa

extern

En variables globales, indica que está está definida en otro lado, pero se la utiliza aquí

`extern`

Todas las funciones son normalmente `extern`, y pueden ser utilizadas por otras partes del programa

Esto es incidentalmente lo que va en un header

```
// Declaramos que 'contador' es una variable entera definida
en otro lugar.
// No se asigna memoria aquí.
extern int contador;

// También declaramos el prototipo de la función definida en
soporte.c
void incrementar_contador();
```

El `extern` es algo que **al enlazar** va a aparecer

```
int main() {
  printf("Valor inicial del contador: %d\n", contador);
  incrementar_contador();
  incrementar_contador();
  // Podemos modificar la variable directamente
  contador = 10;
  printf("Valor del contador modificado desde main: %d\n",
contador);
  return 0;
}
```

main.c (parte 2)

Y en contador.c

```c
#include <stdio.h>

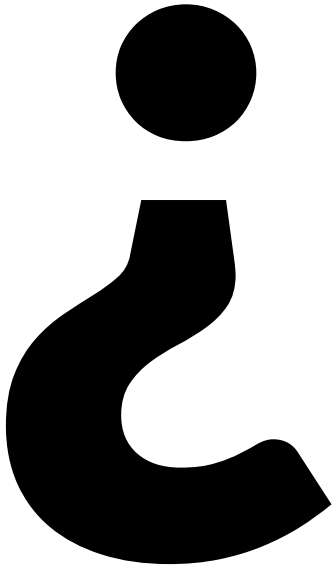
// Definición de la variable global 'contador'.
// Se le asigna memoria y un valor inicial.
int contador = 0;

void incrementar_contador() {
    contador++;
    printf("Contador incrementado a: %d\n", contador);
}
```

static

Aplicado a variables globales o funciones, no admiten que se las *enlace* fuera de donde están definidas.

Limita su alcance.



Qué puede contener



Prototipos de funciones, `matematica.h`

```
int suma(int a, int b);
int resta(int a, int b);
```

Normalmente, **la documentación va acá también**

Constantes de preprocesador

```
#define PI 3.14159
#define CAPACIDAD 300
```

La estructura de un header

Y usando la ‘guarda’

```
#ifndef MATEMATICAS_H
#define MATEMATICAS_H
#define PI 3.14159

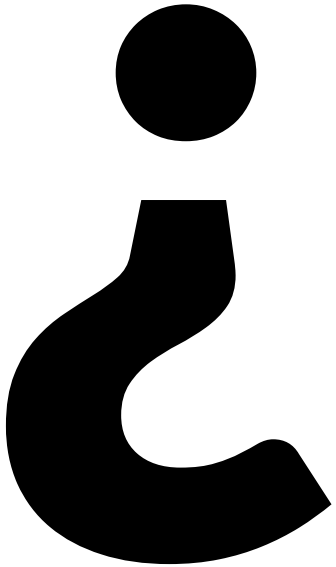
int suma(int a, int b);
int resta(int a, int b);

#endif //MATEMATICAS_H
```

Para evitar que el preprocesador lo inserte múltiples veces

¡A la terminal!





Y como es la compilación



La estructura queda así

```
proyecto
├── main.c
├── matematicas.c
└── matematicas.h
```

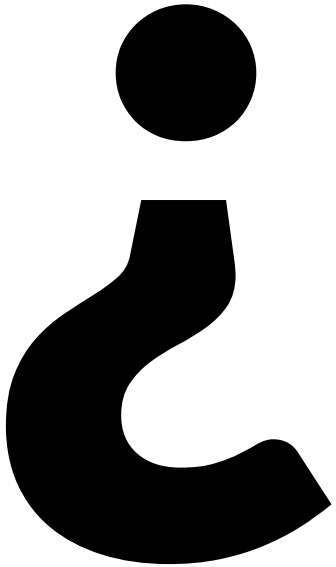
Compilando multiples piezas

```
$> gcc -o binarium main.c matematicas.c
```

Esto crea dos .o y luego los une para construir ‘binarium.exe’

{ Los .h *ya van* con el preprocesador }


¿Preguntas?



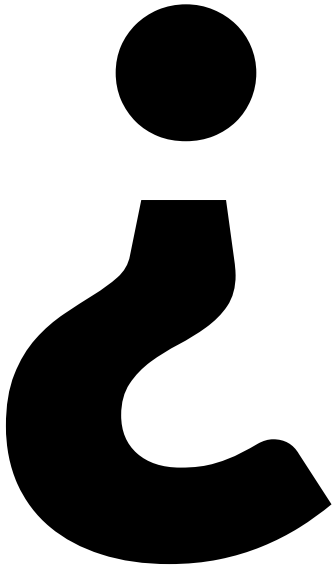
Por que no es lo ‘normal’



{ **Lleva un tiempo extra que en un proyecto grande *suma*** }


¿Preguntas?

Volviendo a lo anterior



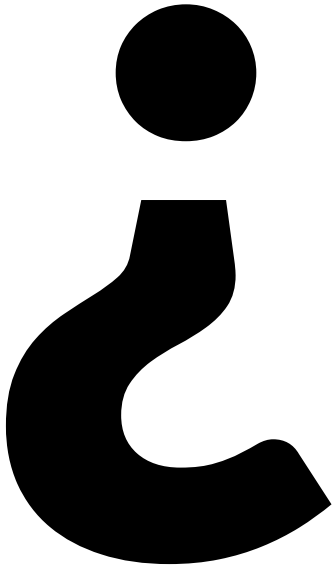
Cada compilación tiene que ser así



¡Nope!

makefiles

UNRN Universidad Nacional de Río Negro



Qué son



make

La receta de cocina para *hornear* programas mas complejos

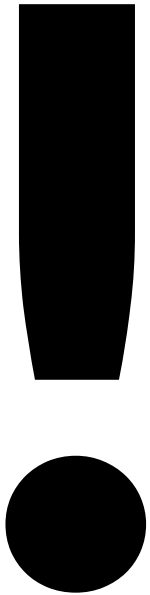
{ Para programas más grandes que un archivo }

Regla básica

```
salida: archivo_entrada
    instrucciones
```



Admite variables



{ Necesita indentación con tabulador } si o si

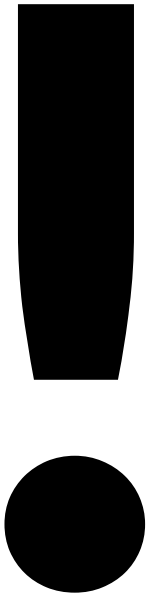
¡A la terminal!



No es *necesario* escribir Makefiles por el momento



Pero hay que entender que pasa



Reglas de compilación en *algún* próximo TP

make

Ejecuta el primer objetivo del `Makefile`, habitualmente usada para solo compilar el programa.

make clean

Usada para limpiar todo lo que se genera por `make`

make test

Regla comúnmente usada para ejecutar pruebas del código en el repositorio.

make run

Regla comúnmente usada para compilar y ejecutar el código en el repositorio.

¡A la terminal!



La estructura de la próxima práctica

Las piezas base

```
├── Makefile          el Makefile general
└── plantilla         un directorio base
    ├── ejercicio.c   el archivo con el código
    ├── ejercicio.h   el encabezado de las funciones
    ├── main.c        donde va el main
    ├── Makefile      el Makefile del ejercicio
    └── prueba.c      el main de pruebas
```

1

No es necesario cambiar los Makefiles

2

Completen la plantilla antes de copiarla para los ejercicios

3

Si quieren cambiar los nombres de los archivos es necesario ajustar el Makefile

¡A la terminal!




¿Preguntas?

Opciones de compilador recomendadas

-Wall

Activa más advertencias de compilación, como variables declaradas y no utilizadas.

Variables sin inicializar

```
int main() {
    int uninitialized;
    int y = 5;

    uninitialized = y + 1;
    printf("El valor de la variable es: %d\n", uninitialized);

    return 0;
}
```

-Wextra

Activa aún más advertencias de compilación

Argumento no utilizado

```
void greet(const char name[]) {
    printf("¡Hola!\n");
}
```

-fanalyzer

Analiza el flujo del programa sin ejecutarlo para encontrar problemas potenciales.

Índice de arreglo ‘contaminado’ (tainted)

```
int data[10];
int user_index;

for (int i = 0; i < 10; ++i) {
  data[i] = i * 2;
}

printf("Por favor, introduce una posición(0-9): ");
scanf("%d", &user_index);
printf("El valor en el índice %d es: %d\n", user_index,
data[user_index]);
```

¡A la terminal!



Macros *integrados* interesantes

__FILE__

1

Es reemplazado por el nombre del archivo siendo compilado.

__LINE__

2

Es reemplazado por la línea de código en la que es llamado

__func__

3

Es reemplazado por el nombre de la funcion actual.

__date__ / __time__

3

Es reemplazado por una cadena con fecha y hora

DEBUG

4

Se define cuando el programa está en desarrollo

```
$> gcc -DDEBUG programa.c
```

```
#ifdef DEBUG
//estamos en desarrollo!
```

¡A la terminal!



Macros avanzados

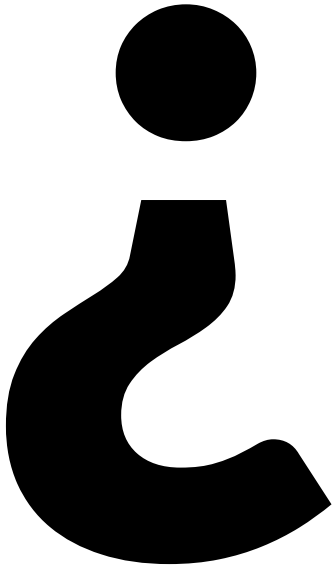
Macros parametrizados

```
#include <stdio.h>

#define SQUARE(x) ((x) * (x))

int main(void) {
    int val = 5;
    printf("El cuadrado de %d es: %d\n", val, SQUARE(val));
    return 0;
}
```

No les vamos a dar uso (complican *mucho* el debug).

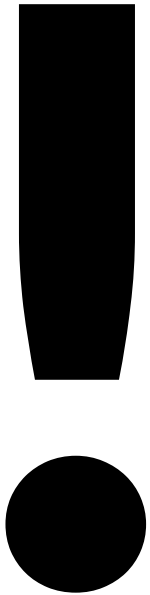


Por qué





Se resuelven en tiempo de compilación



unrn.edu.ar

UNRN Universidad Nacional de Río Negro

| unrionegro